From Traces to Guarded Programs: Evidence-Gated Compilation of Recurrent Agent Workflows

Anonymous authors
Paper under double-blind review

Abstract

Tool-using agents often revisit the same read-only evidence path while paying for a model decision at every tool boundary. Replacing a recurring model-mediated prefix could save these calls, but it is not ordinary caching: a later tool argument may encode a decision unrecoverable from observed state, an apparently read-only call may cross an effect boundary, and correct tool replay may still change the remaining agent's answer. Existing recurrence, replay, and workflow-induction signals therefore identify candidate regions, but do not establish that replacing the model is safe.

We introduce guarded agentic compaction (GAC), a compile-or-retire trace-to-program system that makes this evidence requirement explicit. GAC normalizes recorded episodes, mines recurrent read-only prefixes, and emits a bounded deterministic program only when every tool argument has a provenance witness from entry state or a prior result; an unwitnessed argument retires the region while preserving its maximal justified prefix. Manifest, input, and output guards enforce the resulting contract, and a fixed finite-sample selective-risk test decides deployment; a failed check falls back to the unchanged agent. We evaluate this lifecycle on three live GitHub workflow families, a time-forward five-repository extension, and three refusal benchmarks. On the live families, GAC attains 90/90 exact held-out contracts, versus 89/90 for the unchanged agent, while reducing provider requests by 66.6% and tokens by 63.1% in aggregate. It completes four of five extension repositories and retires one; on all three refusal benchmarks it retires every recurrent family despite successful provenance, synthesis, and replay. GAC thus turns recurrence from permission to rewrite an agent into a testable, evidence-gated specialization decision with principled refusal and fallback.

1 Introduction

The standard agent loop pays for a model decision at every tool boundary: model, tool, model, tool, and so on (Yao et al., 2023). That flexibility is valuable while a workflow is novel. At scale, however, mature agents often revisit the same read-only evidence path with the same data dependencies. In those settings, repeating the model decision adds latency, tokens, and cost without necessarily adding useful adaptation.

Removing that decision is not ordinary caching. A tool argument may depend on a specific earlier observation, a repeated sequence may cross an approval or write, an apparently read-only tool may hide an undeclared effect, and a program that replays the same tool outputs may still change the downstream answer. The problem is therefore not just "find recurring traces." The problem is to decide when a recurrent model-mediated region can be replaced by a deterministic program and when it should be refused.

Consider held-out issue #4420 in the expanded 30-record study. Given only `issue_number=4420`, the observed trace calls `record(4420)` and then passes the *returned* `record.issue_number` to `labels` and to `comments`, which it invokes with `limit=100` before rendering an auditable triage answer. All 132 discovery executions take that route, so a recurrence-only optimizer would emit all three reads. GAC instead retires the *region*: `comments.limit` varies across otherwise similar entries and is reachable neither from the entry state nor from either earlier result, so one unwitnessed slot blocks the whole three-read candidate as `ungroundable_slot`. What survives is the maximal

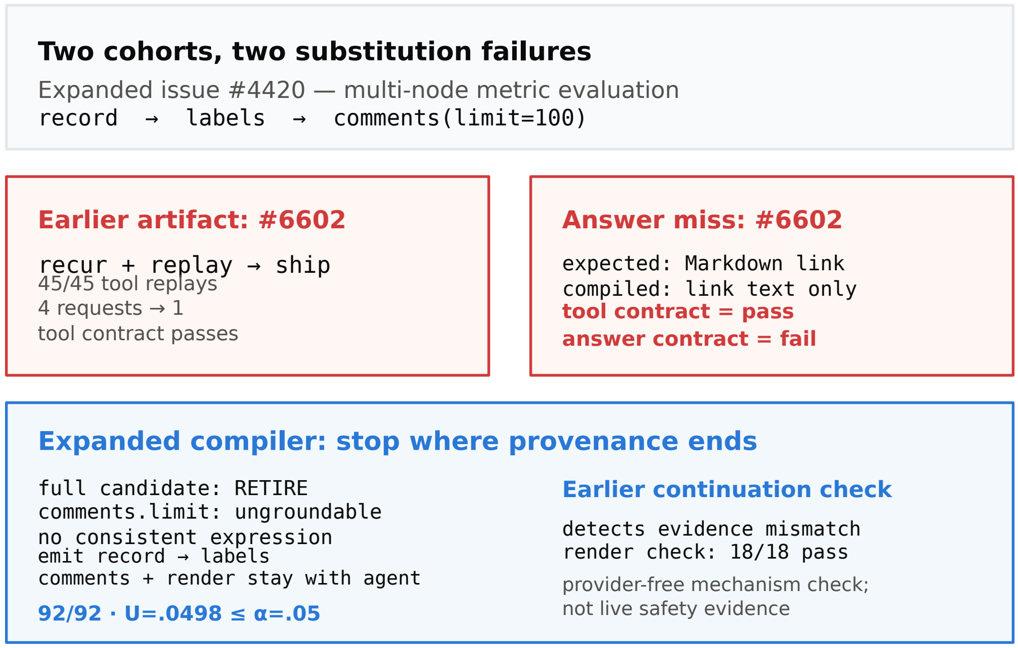

Figure 1: Two retained cohorts expose different failure modes. In the expanded study, issue #4420 forces a shorter grounded prefix; in the earlier study, issue #6602 shows that clean tool replay need not preserve the downstream answer.

justified prefix `record` $\longrightarrow$ `labels`; the agent still chooses the comment limit and writes the answer. That two-read artifact passes 92/92 calibration groups at $\alpha = .05$ (upper bound 0.0498).

Why stop there? A separate, earlier 18-record study supplies the counterexample: its aggressive three-read artifact replayed all 45 tool calls, yet issue #6602 lost the URL from a Markdown link and only 17/18 downstream answers remained exact. A provider-free continuation guard detects and checked-renders that miss; it is a mechanism check, not live safety evidence. Together the two cohorts (fig. 1) show why recurrence proposes a candidate, while provenance, continuation scope, and statistical evidence determine what may replace the model.

Framed this way, the closest precedent is not prompt optimization but the tracing just-in-time compiler. Dynamo (Bala et al., 2000) detects a hot path, compiles it behind a *guard*, and deoptimizes to the interpreter when the guard fails (Hölzle et al., 1992). Agent traces admit the same structure — hot-trace detection becomes family mining, guard synthesis becomes an entry contract, and deoptimization becomes fallback to the unchanged agent — but they add three obligations a JIT does not face. Tool arguments must be shown to be *grounded* in observable state rather than invented by the model; declared *effects* must permit speculation, because an agent's "interpreter" can call the outside world; and the residual error rate of a compiled region is statistical, so it needs a finite-sample bound rather than a soundness proof.

We study these obligations through *guarded agentic compaction* (GAC), a compile-or-retire trace-to-program system for recurrent read-only prefixes (fig. 2). The core contribution is not "agent compilation" in general, but an admissibility argument for trace-derived specialization:

1. A guarded specialization pipeline (Alg. 1) that combines typed provenance, effect and position barriers, bounded synthesis, runtime verification, and finite-sample compile-or-retire admission (Alg. 2, proposition 1); every stage can refuse, and the default output is the unchanged agent.
2. A real-provider evaluation across three public-record workflow families, plus a narrower cross-repository, time-forward extension that keeps both successful artifacts and principled retirements.

3. A negative-result boundary showing that recurrence and replayability do not themselves license deployment: NESTFUL, API-Bank, and executed BFCL gold plans retire under the same admission logic, and hand-written programs are competitive or tied wherever the workflow is already known, so no claim of automatic superiority is warranted.

2 Problem Formulation

Episodes and candidate regions. An episode $E = (z, M, e_{1:T}, y)$ contains an entry-state snapshot z, a content-addressed execution manifest M, ordered events e_t (model requests and responses, tool calls and results, handoffs, guardrails, approvals, errors, commit boundaries), and an observable outcome y. Each tool f carries an application-owned effect declaration $\epsilon(f)$ and capability flags such as *speculatable*; an unknown effect is not a read.

A candidate region $R = [a, b]$ begins at a model-request boundary and ends after a tool result. It is *groundable* when every call argument is either an application-declared literal for that slot or an expression over entry state or prior results inside R,

$$\forall c_j \in R,\ \forall u \in \mathrm{args}(c_j): \quad \mathrm{Lit}(c_j, u)\ \vee\ \left[u = g_{j,u}\big(z, o_{<j}\big), g_{j,u} \in \mathcal{L}\right], \tag{1}$$

where Lit is a schema-level allowlist fixed independently of the traces and $\mathcal{L}$ is a closed, bounded transform library; and *effect-admissible* when every tool in R is declared read-only, speculatable, replayable, and inside one principal and isolation partition. Handoffs, approvals, writes, errors, and unknown effects are hard barriers, not costs to be traded away. Because the deployed runtime resolves artifacts at the initial model boundary, the compiler also imposes a *position invariant*:

$$a = 0 \quad \text{in the normalized model-boundary index.} \tag{2}$$

This is not aesthetic. A suffix program dispatched at entry can reorder or duplicate calls the agent has already made — a semantic mismatch between compilation-time and resolution-time position rather than a tuning failure, and a pilot that violated eq. (2) produced exactly that fault (appendix E).

Selective optimization objective. The compiler emits an artifact $A = (P, H, V, q, \eta, M)$: a deterministic program P, hard guard H, output verifier V, nonconformity score q, threshold η, and manifest pins M. Let $x = (z, M', \omega)$ denote a future entry state, runtime manifest, and external state. Here $M' \simeq M$ means equality on every pinned compatibility field; ω can affect outcomes and cost but is not observable at the dispatch boundary. Dispatch is *selective*,

$$d_A(x) = \mathbf{1}\{\, H(z, M') = 1\ \wedge\ q(z) \le \eta\ \wedge\ M' \simeq M \,\}, \tag{3}$$

and any failed precondition runs the unchanged baseline agent B. If execution fails before a commit and staging proves the attempt clean, B runs; a dirty failure after a commit is an incident, not a fallback. Fix $L(A, x) = 1$ for an externally visible task/contract violation or incident after dispatch; a clean abort followed by unchanged B has $L = 0$. Let $C(A, x)$ include the complete realized cost of that attempt, including a clean fallback. For a future group $G = (x_1, \ldots, x_m)$, define $D_A(G) = \max_i d_A(x_i)$ and $W_A(G) = \max_i d_A(x_i) L(A, x_i)$. We seek

$$\max_{A}\ \mathbb{E}\big[d_A(X)\,\{C(B, X) - C(A, X)\}\big] \quad \text{s.t.} \quad \Pr[W_A(G) = 1 \mid D_A(G) = 1] \le \alpha. \tag{4}$$

Two properties shape everything that follows. The constraint conditions on dispatch: abstention does not enter its denominator, although it earns no savings in the objective; by convention its conditional risk is zero when population dispatch probability is zero. Returning *no* artifact is feasible, so RETIRE is a legitimate output. Equation (4) is a design target: the implementation calibrates a bounded ranked set and retains nondominated survivors, and we prove neither optimality nor a compiler-wide guarantee for that search. Calibration uses independent scenario groups as its statistical units. The group and episode risks coincide in the reported live studies, where every group contains exactly one public record; they need not coincide for unequal multi-episode groups.

3 Guarded Agentic Compaction

Figure 2 and Alg. 1 give the same six stages at two levels of detail. The order is itself part of the argument: each stage can only weaken a candidate, so a later stage may reject one an earlier stage passed, but no later stage can authorize one that an earlier stage blocked. We describe each and state what it can refuse.

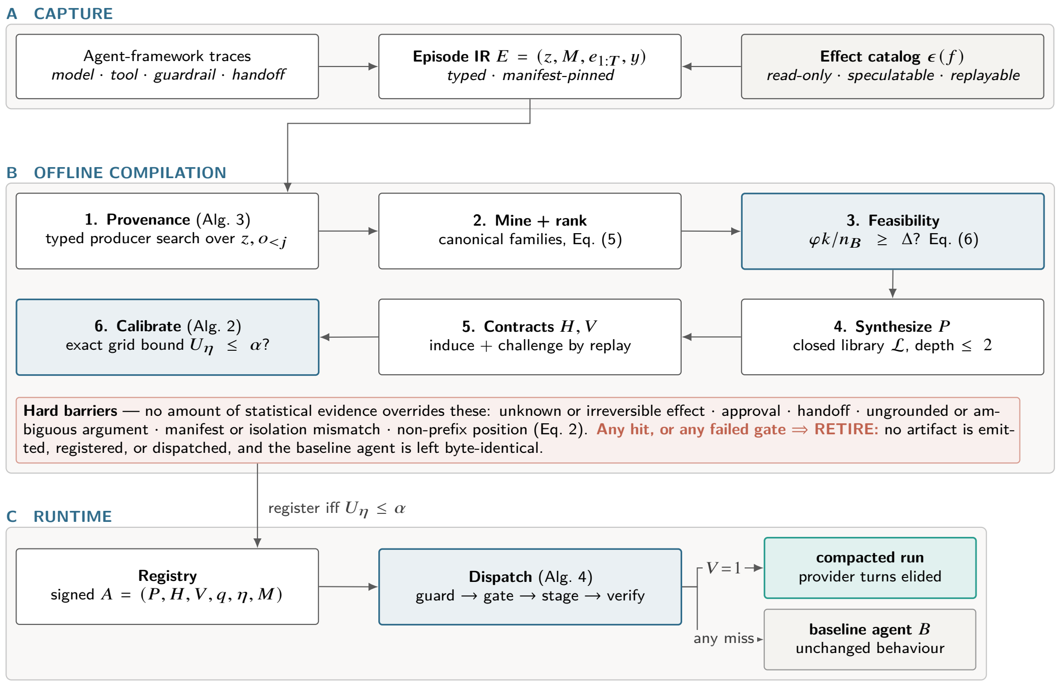

Figure 2: Architecture of GAC. **(A)** Capture normalizes traces into one typed Episode IR against a declared effect catalog. **(B)** Offline compilation is a cascade of independent rejection points; the shaded band lists barriers no statistical evidence can override. **(C)** At runtime exactly one terminal edge compacts; the rest return the unchanged agent.

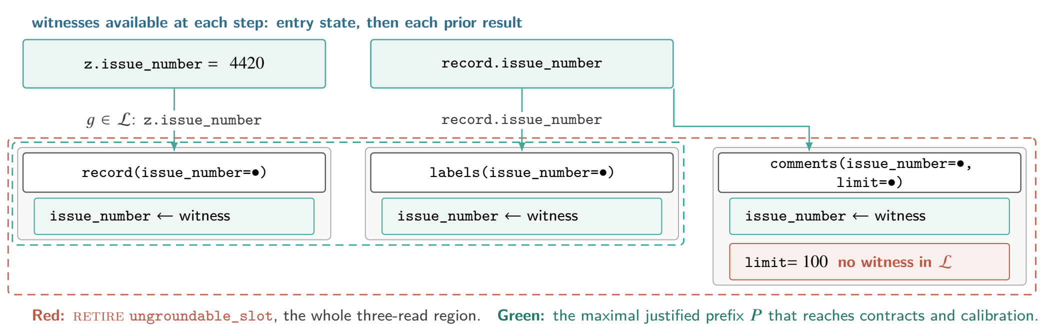

Figure 3: Equation (1) on the held-out issue-4420 record. Every argument slot needs an expression in $\mathcal{L}$ over the entry state or a prior result; the two `issue_number` slots have one and the `limit` slot has none, so it is drawn as a socket with no incoming edge. The question is strictly harder than recurrence: not "did this sequence repeat?" — all 132 discovery executions did — but "can every socket be wired from an allowed prior value?" One unwitnessed slot retires the whole region, and a slot with more than κ witnesses does the same.

1. Typed provenance. A capture adapter normalizes framework traces into the Episode IR; manifests pin prompt, policy, guardrail, tool, model, SDK, tracer, entry-contract, and effect-catalog identities, and episodes with different compatibility keys are partitioned before any learning. For each call-argument slot, the program-argument trace graph (PATG, Alg. 3) searches entry-state and prior-result paths for typed values reachable by a bounded transform — execution provenance in the database sense (Cheney et al., 2009), specialized to tool arguments. Two outcomes block a region rather than guessing: a slot with *no* witness is a genuine model decision, and a slot with more than

Algorithm 1 COMPILE — the compile-or-retire cascade. Every candidate is either registered or assigned an attributed RETIRE, so refusal accounting remains explicit. Stages marked (CALLER) are separate library entry points run in this order rather than steps inside one function.

Input: episodes $\mathcal{E}$; effect catalog C; entry schema Θ; transform library $\mathcal{L}$; ambiguity cap κ; budgets α, δ; grid Λ; feasibility target Δ
Output: artifact set $\mathcal{A}$, or RETIRE with attributed reasons

$\mathcal{A} \leftarrow \emptyset$
$\mathcal{E} \leftarrow \textsc{Qualify}(\mathcal{E}, C)$; $\{\mathcal{E}_M\} \leftarrow \textsc{PartitionBy}(\mathcal{E}, \text{manifest, principal, isolation})$ ▷ drop partial runs, unpinned manifests
for all partitions $\mathcal{E}_M$ **do**
  $(\mathcal{T}, \mathcal{D}, \mathcal{K}, \mathcal{S}) \leftarrow \textsc{GroupedSplit}(\mathcal{E}_M)$ (CALLER) ▷ train / dev / calibration / sealed test, split by *group*
  $G \leftarrow \textsc{BuildPatg}(\mathcal{T}, \Theta, C, \mathcal{L}, \kappa)$ ▷ Alg. 3: block ungrounded or ambiguous slots
  $\mathcal{F} \leftarrow \textsc{Canonicalize}(\textsc{MineRegions}(G, C, w_{\max}))$, keeping only cross-group support
  if $\textsc{Ceiling}(\mathcal{T}, \mathcal{F}) < \Delta$ **then** (CALLER)
    record RETIRE **(infeasible ceiling, Eq. 6)**; **continue** ▷ try the next compatible partition
  end if
  for all families $F \in \mathcal{F}$ ranked by Eq. (5) **do**
    $P \leftarrow \textsc{Synthesize}(F, \mathcal{L})$ at depth ≤ 2; **if** $P = \perp$ **then record** RETIRE **(synthesis); continue**
    $(H, V) \leftarrow \textsc{InduceContract}(F)$; **if** $\neg\textsc{ReplayAndChallenge}(P, V, \mathcal{D})$ **then record** RETIRE **(counterexample); continue**
    $q \leftarrow \textsc{FitScore}(\mathcal{D})$; **freeze** q ▷ dev groups only; never calibration
    $\eta \leftarrow \textsc{Calibrate}(P, H, V, q, M, \mathcal{K}, \Lambda, \alpha, \delta)$; **if** $\eta = \perp$ **then record** RETIRE **(risk); continue** ▷ Alg. 2
    $\mathcal{A} \leftarrow \mathcal{A} \cup \{\textsc{Package}(P, H, V, q, \eta, M)\}$ with evidence and lifecycle
  end for
end for
return $\textsc{SelectUndominated}(\mathcal{A})$ if nonempty, else RETIRE

$\kappa = 3$ witnesses is ambiguous. Figure 3 shows both branches on the motivating record. A repeated tool name, a matching literal, and a successful replay are not witnesses; only an expression in $\mathcal{L}$ is.

2. Region mining and ranking. The miner enumerates model-boundary-to-result windows of at most $w_{\max}$ tool calls, qualifies live-ins and effects against a human-signed rather than inferred catalog, and hashes tool signatures, dependency topology, run shape, and live-in/live-out shape while abstracting literal values. Families require support across independent scenario *groups* rather than raw event frequency, which prevents a single chatty scenario from manufacturing recurrence. Survivors are ranked by an MDL-inspired trade-off that pays for expected savings and charges for heterogeneity, effect exposure, and program size:

$$\text{score}(F) = \underbrace{s_F\, \bar{k}_F\, c_m}_{\text{support-weighted saving}} - \lambda_1 \mathsf{Ent}(F) - \lambda_2 \rho_{\text{eff}}(F) - \lambda_3 |F|, \tag{5}$$

with s_F the number of supporting train groups, $\bar{k}_F$ the mean removable model requests, c_m their unit cost, and $\mathsf{Ent}(F)$ the Shannon entropy over canonical variants inside the family; without λ_1 the ranking prefers a heterogeneous family whose single canonical form fits none of its members. Equation (5) affects only *which* family is tried first — no admission decision depends on it.

3. Feasibility ceiling. Before any synthesis runs, an estimator bounds what the compiler could achieve with a perfect verifier and a gate that never abstains:

$$\Delta_{\max} = \frac{\varphi\, k}{n_B}, \qquad \text{necessary:} \quad \varphi\, \rho\, k \;\geq\; \Delta\, n_B, \tag{6}$$

with n_B baseline model requests per episode, φ the fraction of episodes holding an eligible region, k the requests removed per successful dispatch, and ρ the verifier pass rate. Since eq. (6) assumes $\rho = 1$, no measured reduction can exceed it, and it is met with equality exactly when nothing abstains and nothing fails. Reporting it first makes a decisive negative answer cheap: a workload whose ceiling sits under the target can be declined without building a compiler for it.

4–5. Bounded synthesis and contracts. Synthesis searches a 23-operator library at depth at most two, covering path selection, indexing, string normalization, arithmetic, comparisons, bounded collection operations, and narrow loops; bindings are ranked by stability across groups, and a group-wise

Algorithm 2 CALIBRATE — fixed-grid exact selective admission. The score q and the grid Λ are frozen *before* this procedure sees a calibration group, which is what makes the union bound legitimate.

Input: frozen candidate $A_0 = (P, H, V, q, M)$ without threshold; calibration groups $\mathcal{K}$ with fixed per-instance violations $L(A_0, x)$; grid Λ; budgets α, δ
Output: threshold η with a simultaneous guarantee, or $\perp$ (RETIRE)

- $\mathcal{A} \leftarrow \emptyset$ ▷ admissible (coverage, threshold) pairs
- **for all** $\eta \in \Lambda$ **do**
  - define $d^\eta(x)$ by Eq. (3) with threshold η; $D^\eta(g) \leftarrow \max_{x \in g} d^\eta(x)$; $W^\eta(g) \leftarrow \max_{x \in g} d^\eta(x) L(A_0, x)$
  - $K_\eta \leftarrow \{ g \in \mathcal{K} : D^\eta(g) = 1 \}$; $n_\eta \leftarrow |K_\eta|$; **if** $n_\eta = 0$ **then continue** ▷ vacuous bound $U_\eta = 1$
  - $k_\eta \leftarrow |\{ g \in K_\eta : W^\eta(g) = 1 \}|$ ▷ *wrong after dispatch* only, never abstention
  - $U_\eta \leftarrow 1$ if $k_\eta = n_\eta$, else $\mathrm{Beta}^{-1}\big(1 - \frac{\delta}{|\Lambda|};\ k_\eta + 1,\ n_\eta - k_\eta\big)$; **if** $U_\eta \le \alpha$ **then** $\mathcal{A} \leftarrow \mathcal{A} \cup \{(n_\eta / |\mathcal{K}|, \eta)\}$ ▷ Eq. (7)
- **end for**
- **return** $\perp$ if $\mathcal{A} = \emptyset$, else the η component of $\mathrm{lexmax}_{(\mathrm{cov}, \eta) \in \mathcal{A}} (\mathrm{cov}, \eta)$ ▷ largest empirical group coverage; larger threshold breaks ties

refit rejects bindings that exploit row-level coincidence. The hypothesis space is deliberately small enough to read: the approach resembles programming-by-example (Gulwani, 2011) and bounded sketching (Solar-Lezama et al., 2006), and it does *not* generate arbitrary Python. Hard guards then constrain entry types, required fields, categorical sets, numeric intervals, patterns, isolation keys, allowed effects, and manifest pins; verifiers constrain live-out presence, type, cardinality, empirical hulls, provenance, effect multisets, and call counts. These are candidate invariants, not proofs for all future inputs — which is why a statistical gate follows them rather than replacing them.

6. Exact risk-gated admission. Algorithm 2 decides deployment. The score q is fitted on *dev* groups only — never on calibration groups — from entry-observable features such as missing fields, hull margin, temporal drift, and provenance ambiguity, then frozen together with the finite threshold grid $\Lambda = \{.02, .05, .08, .11, .14, .17, .20, .25, .30, .40, .50\}$. Two data signals do two different jobs here, and conflating them is the easiest way to misread the guarantee: q is *fitted* on development groups that turned out unproductive in any way — wrong or abstained — while the bound below is *counted* only from groups that were dispatched and turned out wrong. For each $\eta \in \Lambda$ the compiler counts admitted calibration groups (n_η) and those containing at least one post-dispatch violation (k_η), and inverts the one-sided exact binomial test (Clopper & Pearson, 1934) at a Bonferroni-corrected level, with explicit empty and all-violation cases:

$$U_\eta = \begin{cases} 1, & n_\eta = 0 \text{ or } k_\eta = n_\eta, \\ \mathrm{Beta}^{-1}\Big(1 - \frac{\delta}{|\Lambda|}; k_\eta + 1, n_\eta - k_\eta\Big), & 0 \le k_\eta < n_\eta, \end{cases} \qquad U_\eta\Big|_{k_\eta = 0} = 1 - \Big(\frac{\delta}{|\Lambda|}\Big)^{1/n_\eta}. \quad (7)$$

It admits the largest-coverage threshold with $U_\eta \le \alpha$ and retires the family otherwise — a fixed-grid instance of the learn-then-test discipline (Angelopoulos et al., 2022). The closed form on the right is the operative constraint in our experiments: at $\alpha = 0.05$, $\delta = 0.1$, and $|\Lambda| = 11$, a zero-violation family still needs $n_\eta \ge 92$ admitted groups. The default output is refusal, not emission.

Proposition 1 (Per-candidate selective-risk admission)**.** *Condition on train/development data and fix one candidate (P, H, V, q), group-level violation rule, and finite grid Λ before observing calibration. If calibration groups are i.i.d. from the future-group distribution, then with probability at least $1 - \delta$, every $\eta \in \Lambda$ satisfies $r_\eta \le U_\eta$, where r_η is $\Pr(W^\eta = 1 \mid D^\eta = 1)$ when population admission is positive and is defined as zero otherwise. Consequently the selected $\hat{\eta}$ satisfies $r_{\hat{\eta}} \le \alpha$ whenever $U_{\hat{\eta}} \le \alpha$.*

Appendix A gives the proof. The guarantee is deliberately narrow: it is *per fixed candidate*, not compiler-wide over the adaptive candidate search Alg. 1 actually performs, and it conditions on group exchangeability rather than establishing it (section 7). The scientific point is that guarded deployment requires an explicit evidence boundary, not just a recurrent trace and a successful replay.

Table 1: Primary GitHub benchmark at a glance. Each sample supplies one record number from a pinned public snapshot; every listed answer field is checked against the source record.

Task	Read-only evidence path	Exact answer contract
Issue type	`issue_number` → record, official labels, comments	Title, state, label-derived category, verbatim excerpt
PR outcome	`pr_number` → PR record, merge status, discussion	`open`, `merged`, or `closed_unmerged`, plus title and comment excerpt
Backlog attention	`issue_number` → open-issue record, assignees, discussion	`owned`, `discussed_unowned`, or `awaiting_first_response`, plus title, owner, and excerpt

4 Experimental Setup

Because the claim is an *admissibility* claim, the evaluation asks three questions: does specialization preserve outcomes while saving work on real records (section 5.1)? Does it transfer under a time-forward split (section 5.2)? And does it refuse when evidence is thin (section 5.3)?

4.1 Benchmark at a glance

Every GitHub sample begins with one issue or pull-request number from a pinned public snapshot. The agent must make read-only calls and return exact, source-grounded fields, not a subjective preference. The three primary families use 132 discovery and 30 balanced held-out records each, with live provider calls. Table 1 shows what one sample asks an agent to read and return; it tests both the structured answer and whether its evidence remains available after compaction.

Cross-repository time-forward extension. A narrower two-read PR-outcome task spans five frozen public repositories. Held-out records are strictly newer than discovery records, and selection is sealed before provider calls; repositories without an admitted artifact remain in the result. A balanced rerun on three repositories controls the `open`/`merged`/`closed_unmerged` mix.

External refusal benchmarks. NESTFUL and API-Bank supply valid executable traces whose evidence is still insufficient; they retain the argument-level producer structure that provenance, synthesis, replay, and admission need, and they measure none of them as live planning quality (Basu et al., 2025; Li et al., 2023). Executed BFCL v4 gold plans add a third such substrate (appendix F.3).

Conditions, metrics, and statistics. Each GitHub family compares the unchanged baseline B, the compiled artifact, and a hand-written pre-model program on the same records, so the comparison is paired rather than across cohorts. Quality is exact task-contract satisfaction; resources are requests, interfaces, tokens, latency, and estimated cost. We use paired exact McNemar tests, bootstrap intervals, and Wilcoxon signed-rank tests; the only guaranteed multiplicity control is the Bonferroni term in eq. (7). All admission runs use $\alpha = 0.05$, $\delta = 0.1$, and $|\Lambda| = 11$ (appendix C).

5 Results

5.1 Primary workflows: preserved contracts, less work

Across the 90 held-out records, GAC satisfies 90/90 exact contracts versus 89/90 for the unchanged agent (table 2). It reduces provider requests by 66.6%, tokens by 63.1%, latency by 64.2%, and estimated cost by 58.7% in aggregate. The per-family pattern is intuitive: issue-type routing admits only a two-read prefix, whereas the two deeper workflows approach 75% fewer requests.

No held-out record is a compiled-only failure; the one-sided 95% upper bound on that discordance is 3.3%. This supports preservation on the observed cohort, not equivalence in general. Each artifact was admitted with 92 zero-violation calibration groups ($U_\eta = 0.0498 \leq .05$). Hand-written programs also achieve 90/90 and are cheaper for issue-type routing, so the evidence supports automatic discovery and guarded lifecycle management—not runtime dominance over correct manual code.

Table 2: Primary live-provider results (n = 30 held-out records per family). "Interfaces" counts provider-visible tool interfaces. All conditions see identical records, model, and grader; *Manual* is hand-written with full knowledge of the workflow.

	Exact held-out contract			Reduction of compiled vs. baseline (%)				
Family	Base	Compiled	Manual	Requests	Interfaces	Tokens	Latency	Cost
Issue-type routing	30/30	**30/30**	30/30	50.0	0.0	39.5	51.7	32.0
PR-outcome audit	30/30	**30/30**	30/30	75.0	66.7	80.8	73.0	75.3
Backlog-attention routing	29/30	**30/30**	30/30	74.8	66.3	81.4	68.9	75.1
Weighted total (n = 90)	89/90	**90/90**	90/90	66.6	44.2	63.1	64.2	58.7

Table 3: Cross-repository time-forward results on the narrower two-read pull-request outcome task. Held-out records are strictly newer than discovery records and selection is sealed before any provider call. The 5-repo cohort retires `pytorch/pytorch`. A fixed two-read template is more efficient in the 5-repo core (66.7% fewer requests against 44.4%) and essentially tied in the balanced rerun, which is why this result is evidence of automatic discovery and principled retirement rather than of runtime dominance.

		Exact task contract			Reduction vs. baseline (%)			
Setting	Repos	Discovery	Held-out	Classes	Req.	Tokens	Latency	Cost
5-repo core	4/5	580/580	120/120	skewed	44.4	52.4	49.4	48.6
3-repo rerun (balanced)	3/3	360/360	180/180	balanced	66.7	78.4	60.7	72.7

Table 4: Where refusal happens. All three benchmarks clear provenance, synthesis, and replay with *zero* wrong held-out executions, then retire at calibration: sample size binds, not correctness. Stages refer to Alg. 1; all three need 92 zero-violation groups by Eq. (7).

Stage of Alg. 1	NESTFUL	API-Bank	BFCL v4
Episodes / recorded traces	1,415	212	200
Tools compilable / effect-blocked	—	21 / 28	34 / 47
Dependency slots with producer recovered (1)	5,531 / 5,746 (96.3%)	559 / 881 (63.5%)	1,445 / 2,124 (68.0%)
Episodes yielding an admissible window (2)	1,207 (85.3%)	37 (17.5%)	86 (43.0%)
Families at or above the support floor (2)	32	8	9
Families that synthesize a program (4)	12	2	4
Held-out replay: pass / abstain / **wrong** (5)	24 / 12 / **0**	0 / 2 / **0**	3 / 3 / **0**
Largest family support $n_{\max}$ (6)	26	8	15
Artifacts admitted	**0**	**0**	**0**

5.2 Time-forward transfer: compile or retire

In the five-repository cohort, GAC completes 120/120 exact held-out pairs on four repositories and retires `pytorch/pytorch` at compile time rather than loosening its gate (table 3); the balanced three-repository rerun reaches 180/180 exact pairs with 66.7% fewer requests.

5.3 Refusal when evidence is insufficient

The external benchmarks expose the opposite outcome, stage by stage in table 4. All three clear provenance, synthesis, and held-out replay with *no* wrong execution anywhere, and all three still retire: their largest families reach 26, 8, and 15 groups against the 92 that eq. (7) requires. Executed BFCL gold plans are the pre-registered case: their outcome was recorded before the compiler saw them.

All three therefore separate trace-level properties from deployment evidence: provenance reachability and replay correctness identify a plausible program without certifying that it should replace a model.

6 Related Work

Guarded trace and agent workflow compilation. GAC adapts the tracing-JIT pattern—compile a hot path behind a guard and return to the interpreter on a miss (Bala et al., 2000; Hölzle et al., 1992)—to agents, adding grounded arguments and declared effects (Lucassen & Gifford, 1988). Agent systems mine meta-tools or workflows, cache plans, compile orchestration, validate plans, search graphs, or prune topology (Abuzakuk et al., 2026; Wang et al., 2025; Zhang et al., 2025; Wei et al., 2026; Winston et al., 2026; Li et al., 2026; Chen et al., 2026). DSPy optimizes prompts, GEPA evolves them, RouteLLM changes models, and LLMCompiler parallelizes current calls (Khattab et al., 2024; Agrawal et al., 2026; Ong et al., 2025; Kim et al., 2024). Unlike these targets, GAC treats recurrence only as a proposal: provenance, effects, position, contracts, and finite-sample risk each retain a veto.

Synthesis, risk control, and evaluation. Bounded search draws on programming by example, sketching, and provenance (Gulwani, 2011; Solar-Lezama et al., 2006; Cheney et al., 2009); admission combines Clopper–Pearson with fixed-grid learn-then-test (Clopper & Pearson, 1934; Angelopoulos et al., 2022). BFCL and ToolLLM score tool use broadly (Patil et al., 2025; Qin et al., 2023), while NESTFUL and API-Bank retain the references and results needed for provenance and refusal (Basu et al., 2025; Li et al., 2023).

7 Discussion and Limitations

The supported claim is narrow: repeated read-only prefixes can become guarded programs that preserve held-out contracts on the tested GitHub families and retire when evidence is inadequate.

Gate maturity is the main scientific risk. Proposition 1 is per-candidate, not compiler-wide over the adaptive search of Alg. 1; certification requires freezing the candidate relative to calibration, and a Bonferroni repair over m candidates would use $\gamma = \delta/(m|\Lambda|)$, raising the zero-violation requirement from 92 admitted groups to 106 at $m = 2$. The gate is also empirically step-like at $n_\eta \geq 92$: we show that it *refuses* correctly far better than that q *discriminates*. Refitting q for a genuine frontier is the key follow-up.

Group independence is assumed, not established. Proposition 1 needs i.i.d. groups. Here each group is one record from one snapshot with `min_days=min_principals=1`; the grouped split does not establish exchangeability under shared schema, community, or time. The calibration cohort must also be sampled before filtering on whether the baseline happened to exhibit the mined region, with violation labels defined for every dispatch-eligible member; post-selecting only recurrent successes would not satisfy the proposition.

External validity and practical competitiveness. The richer evidence comes from one revision-pinned repository and one provider/model family, and the cross-repository extension is a narrower, template-tied task. We establish no superiority under drift or across providers, and do not price manual authoring or maintenance.

Semantic scope, and where "unchanged baseline" is exact. Exact task contracts do not certify a downstream model continuation. Writes, handoffs, hosted tools, undeclared effects, and removed-turn guardrails lie outside proposition 1. Pre-emission rejection is exact on both paths; only a staging-owning runner can restore byte-identical model input after an attempt (appendix B).

8 Conclusion

GAC treats recurrence as candidate evidence, not permission to rewrite an agent. Across live GitHub families and a cross-repository extension it saves resources while preserving exact held-out contracts; on three external benchmarks it refuses. Judge workflow optimizers by their evidence, refusals, and fallback—not only the turns they remove.

AI Use Statement

The authors used generative AI tools to restructure and copy-edit the manuscript, suggest section organization, draft and revise paper text, critique methodological exposition and internal consistency, and convert repository-grounded results into LaTeX tables. They were not used to generate experimental results or to replace manual verification. Every reported number, citation, equation, and comparison was checked against the repository artifacts and retained outputs, and the authors take responsibility for the final content.

Ethics Statement

The main risk in removing model-mediated decisions is over-trusting a compiled prefix, and thereby accelerating unsafe automation or deleting a guardrail evaluation that happened to run at the removed boundary. The method mitigates this by compiling only read-only prefixes, treating unknown effects, approvals, handoffs, and writes as hard barriers, and defaulting to retirement or fallback when evidence is insufficient. All study data are public records (GitHub repository metadata) or public benchmarks; no personal or private data were collected. The artifact is not a license to bypass human review, legal or compliance controls, or safety guardrails; unsupported regimes are part of the result, not exceptions to hide.

Reproducibility Statement

Algorithms 1 and 2 give the compiler and admission protocols, Algs. 3 and 4 give provenance construction and runtime dispatch, appendix A proves proposition 1, and appendix C records the registered statistical and synthesis settings. The accompanying artifact contains source manifests, raw result files, deterministic table and figure generation, and the scripts for the live GitHub studies, the cross-repository extension, NESTFUL, and API-Bank. Before upload, these materials will be supplied as an anonymous archive with hashes and manifests preserved.

References

Sami Abuzakuk, Anne-Marie Kermarrec, Rishi Sharma, Rasmus Moorits Veski, and Martijn de Vos. Optimizing agentic workflows using meta-tools. *arXiv preprint arXiv:2601.22037*, 2026. URL `https://arxiv.org/abs/2601.22037`.

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*, 2026. URL `https://arxiv.org/abs/2507.19457`. Oral presentation.

Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control. In *International Conference on Learning Representations*, 2022. URL `https://openreview.net/forum?id=TNcOrox3tQ`.

Vasanth Bala, Evelyn Duesterwald, and Sanjeev Banerjia. Dynamo: A transparent dynamic optimization system. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 1–12, 2000. doi: 10.1145/349299.349303.

Kinjal Basu, Ibrahim Abdelaziz, Kiran Kate, Mayank Agarwal, Maxwell Crouse, Yara Rizk, Kelsey Bradford, Asim Munawar, Sadhana Kumaravel, Saurabh Goyal, Xin Wang, Luis A. Lastras, and Pavan Kapanipathi. NESTFUL: A benchmark for evaluating LLMs on nested sequences of API calls. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 33538–33547, 2025. doi: 10.18653/v1/2025.emnlp-main.1702. URL `https://aclanthology.org/2025.emnlp-main.1702/`.

Yulang Chen, Haoxuan Peng, Jinyan Liu, Zichen Wen, Dongrui Liu, and Linfeng Zhang. AgentSlimming: Towards efficient and cost-aware multi-agent systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pp. 30064–30086, 2026. doi: 10.18653/v1/2026.acl-long.1387. URL `https://aclanthology.org/2026.acl-long.1387/`.

James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009. doi: 10.1561/1900000006.

C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404.

Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pp. 317–330, 2011. doi: 10.1145/1926385.1926423.

Urs Hölzle, Craig Chambers, and David Ungar. Debugging optimized code with dynamic deoptimization. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 32–43, 1992. doi: 10.1145/143095.143114.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=sY5N0zY5Od`.

Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An LLM compiler for parallel function calling. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 24370–24391, 2024. URL `https://proceedings.mlr.press/v235/kim24y.html`.

Junyan Li, Zhang-Wei Hong, Maohao Shen, Yang Zhang, and Chuang Gan. FlowCompile: An optimizing compiler for structured LLM workflows. *arXiv preprint arXiv:2605.13647*, 2026. URL `https://arxiv.org/abs/2605.13647`.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 3102–3116, 2023. doi: 10.18653/v1/2023.emnlp-main.187. URL `https://aclanthology.org/2023.emnlp-main.187/`.

John M. Lucassen and David K. Gifford. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pp. 47–57, 1988. doi: 10.1145/73560.73564.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. In *International Conference on Learning Representations*, 2025. URL `https://proceedings.iclr.cc/paper_files/paper/2025/hash/5503a7c69d48a2f86fc00b3dc09de686-Abstract-Conference.html`.

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48371–48392, 2025. URL `https://proceedings.mlr.press/v267/patil25a.html`.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023. URL `https://arxiv.org/abs/2307.16789`.

Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 404–415, 2006. doi: 10.1145/1168857.1168907.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 63897–63911, 2025. URL `https://proceedings.mlr.press/v267/wang25bx.html`.

Lei Wei, Qi Liu, Ruiyang Huang, Xiao Peng, Sicong Xie, Lanbo Lin, Chenhao Jiang, Yuanwu Xu, Tianyuan Yang, Jiayao Liu, Li Cai, Zhaolu Kang, and Bin Wang. EvoC2F: Compiling tool orchestration for efficient and evolvable LLM agents. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL `https://openreview.net/forum?id=ZSGB91kMOG`.

Caleb Winston, Ron Yifeng Wang, Azalia Mirhoseini, and Christos Kozyrakis. Agent JIT compilation for latency-optimizing web agent planning and scheduling. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL `https://arxiv.org/abs/2605.21470`.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL `https://openreview.net/forum?id=WE_vluYUL-X`.

Qizheng Zhang, Michael Wornow, and Kunle Olukotun. Cost-efficient serving of LLM agents via test-time plan caching. *arXiv preprint arXiv:2506.14852*, 2025. URL `https://arxiv.org/abs/2506.14852`.

Algorithm 3 BuildPatg — typed argument provenance. A slot with no witness is a genuine model decision; a slot with too many witnesses is ambiguous. Both block the region rather than defaulting to the cheapest explanation.

Input: episodes $\mathcal{T}$; entry schema Θ; effect catalog C; transform library $\mathcal{L}$; ambiguity cap κ
Output: provenance graph G retaining $1, \dots, \kappa$ candidate edges per nonliteral grounded slot; declared literals are marked separately

$G \leftarrow \emptyset$
for all episodes $E = (z, M, e_{1:T}, y) \in \mathcal{T}$ **do**
  $\Sigma \leftarrow \{(\text{“}z\text{”}, \pi, v) : (\pi, v) \in \textsc{Flatten}(z),\ \pi \in \Theta\}$ ▷ only allowlisted entry paths are eligible
  for all tool calls $c_j \in e_{1:T}$ in order **do**
    for all argument slots $u \in \text{args}(c_j)$ **do**
      if u is declared *literal-only* for c_j **then**
        mark (c_j, u) literal; **continue** ▷ preserve the declared constant; infer no provenance
      end if
      $\mathcal{H} \leftarrow \textsc{BestPerSource}\{\, (\sigma, g) \in \Sigma \times \mathcal{L} : g(\text{value}(\sigma)) = u, \text{depth}(g) \le 2 \,\}$
      if $\mathcal{H} = \emptyset$ **then**
        mark (c_j, u) model-originated; **block** region
      else if $|\mathcal{H}| > \kappa$ **then**
        mark (c_j, u) ambiguous; **block** region
      else
        $G \leftarrow G \cup \{\sigma \xrightarrow{g} (c_j, u) : (\sigma, g) \in \mathcal{H}\}$ ▷ synthesis chooses a stable source across groups
      end if
    end for
    $\Sigma \leftarrow \Sigma \cup \{(\text{var}(c_j), \pi, v) : (\pi, v) \in \textsc{Flatten}(o_j),\ \textsc{Eligible}(\pi, v)\}$ ▷ a result witnesses only *later* slots
  end for
  for all pairs (e_s, e_t) sharing a resource where either writes it **do**
    $G \leftarrow G \cup \{e_s \prec e_t\}$ ▷ order edge: no reordering is ever proposed across a conflict
  end for
end for
return G

A Proof of Proposition 1

Condition on the train/development data, fix $\eta \in \Lambda$, and write $\gamma = \delta/|\Lambda|$. Let D_i^η and W_i^η be the group admission and violation indicators defined in section 2. Because the entire candidate and grid were frozen before calibration and groups are i.i.d., the pairs (D_i^η, W_i^η) are i.i.d. Conditional on $n_\eta = m > 0$ admitted groups, their violation count therefore satisfies $k_\eta \mid n_\eta = m \sim \text{Binomial}(m, r_\eta)$, where $r_\eta = \Pr(W^\eta = 1 \mid D^\eta = 1)$. Inverting the one-sided exact binomial test (Clopper & Pearson, 1934) gives the upper bound $U_\eta = \text{Beta}^{-1}(1-\gamma;\ k_\eta + 1,\ m - k_\eta)$ of eq. (7), which satisfies $\Pr\{r_\eta > U_\eta \mid n_\eta = m\} \le \gamma$. When $m = 0$ the event is impossible under the convention that empty-admission thresholds receive the vacuous bound $U_\eta = 1$. Averaging over the random value of n_η therefore yields $\Pr\{r_\eta > U_\eta\} \le \gamma$ for each fixed threshold. Applying the union bound over the finite grid gives $\Pr\{\exists \eta \in \Lambda : r_\eta > U_\eta\} \le |\Lambda|\gamma = \delta$, i.e. simultaneous coverage with probability at least $1 - \delta$. On that event, every threshold whose upper bound is at most α satisfies the group-level target in eq. (4), including the threshold returned by Alg. 2. ■

Two scope conditions are worth restating. First, the statement is *per fixed candidate*: Alg. 1 may calibrate several families and retain survivors, and that outer search is not multiplicity-corrected. Second, i.i.d. source groups is an assumption about the calibration cohort, not a property the compiler verifies; the grouped split in Alg. 1 makes it plausible by construction but does not establish it under temporal drift.

B Additional Algorithms

Algorithm 3 constructs the typed provenance graph that decides eq. (1). Its two block conditions are asymmetric on purpose: a slot with no witness is treated as a genuine model decision and a slot with more than κ witnesses is treated as ambiguous, so the analysis never resolves an argument by picking the cheapest available explanation.

Algorithm 4 DISPATCH — boundary-time admission. Cheap deterministic checks reject first and the calibrated gate runs only on what survives them. Exactly one exit path compacts; clean rejections return the unmodified agent, while an unattested or post-commit failure raises an incident.

Input: entry state z; runtime context M'; registry $\mathcal{R}$; effect catalog C; mode $m \in \{\mathsf{off}, \mathsf{shadow}, \mathsf{live}\}$
Output: observations for the region, FALLBACK to baseline agent B, or INCIDENT

if $m = \mathsf{off}$ **then**
  return FALLBACK ▷ conformance: model-visible input must be byte-identical
end if
$A \leftarrow \mathcal{R}$.RESOLVE(M'.compatibility, M'.partition)
if $A = \perp \lor A$.lifecycle $\neq$ ACTIVE $\lor \neg$VERIFYSIGNATURE(A) **then**
  return FALLBACK
end if
if $H_A(z, M') \neq 1$ **then**
  return FALLBACK ▷ manifest pins, isolation keys, typed hulls, effect set
end if
if tools($A.P$) $\cap$ ALREADYOBSERVED $\neq \emptyset$ **then**
  return FALLBACK ▷ the region already ran; its live-ins are not the fitted ones
end if
if $m = \mathsf{live} \land \exists f \in$ tools($A.P$) : $C(f)$.*quota_attested* $\land$ SNAPSHOT $= \perp$ **then**
  return FALLBACK ▷ no staging owner, so a discarded attempt is not attestable
end if
if $q_A(z) > \eta_A$ **then**
  return FALLBACK ▷ calibrated abstention on an uncertain input
end if
if $m = \mathsf{shadow}$ **then**
  log would-dispatch; **return** FALLBACK ▷ observe coverage without changing behaviour
end if
$S \leftarrow$ STAGE.BEGIN(); $r \leftarrow$ INTERPRET($A.P, z$, FACADE(C)) ▷ the facade re-checks effects at execution time
if r raised PRECOMMITERROR **then**
  return FALLBACK if S.ABORTANDATTEST(), else INCIDENT
else if r raised POSTCOMMITERROR **then**
  return INCIDENT ▷ an external commitment happened; do not feign rollback
end if
if $V_A(r) \neq 1$ **then**
  return FALLBACK if S.ABORTANDATTEST(), else INCIDENT ▷ live-outs, effect multiset, and call count are checked before use
end if
return r.observations if S.COMMIT(r.effects) succeeds, else INCIDENT

Algorithm 4 is the runtime counterpart of eq. (3). Cheap deterministic checks reject first and the calibrated gate runs only on what survives them, so the common case costs a registry lookup and a guard evaluation. Exactly one exit path compacts; every clean rejection returns the unmodified baseline agent, and any failure whose reversibility cannot be attested raises an incident rather than feigning a rollback. "Baseline" is byte-exact only where a staging owner holds the commit boundary: the outer runner can restore byte-identical model input, whereas a model-boundary adapter cannot un-emit a response the host has already committed to conversation history.

C CONFIGURATION

All reported admission runs use risk budget $\alpha = 0.05$, confidence budget $\delta = 0.1$, and a fixed grid of $|\Lambda| = 11$ thresholds, which fixes the zero-violation sample requirement at $n_\eta \geq \log(\delta/|\Lambda|)/\log(1-\alpha) = 91.6$, i.e. 92 admitted groups; at exactly $n_\eta = 92$ and $k_\eta = 0$ the bound is $U_\eta = 1 - (0.1/11)^{1/92} = 0.0498 \leq \alpha$. Tightening the confidence budget to $\delta = 0.05$ would raise the requirement to 106 groups, which no study in this paper reaches, so $\delta = 0.1$ is the operative setting throughout and is reported as such rather than rounded to match α. A Bonferroni repair over $m = 2$ candidate families would use $\gamma = \delta/(m|\Lambda|)$ and land on the same 106-group requirement, which is why the paper reports per-candidate certificates rather than a compiler-wide one. The grid is $\Lambda = \{.02, .05, .08, .11, .14, .17, .20, .25, .30, .40, .50\}$ and the ambiguity cap is $\kappa = 3$. Synthesis uses a 23-operator closed library at depth ≤ 2; region mining uses $w_{\max}$ tool calls per window and requires support across independent scenario groups. The implementation also supports a minimum-distinct-days requirement, which the single-snapshot live study does not exercise (`min_days` = 1). Two terms of eq. (5) are coarser in the implementation than the notation suggests: ρ_{eff} is approximated by the fraction of tool names containing a namespace separator rather than read from the effect catalog, and $|F|$ is substituted by the argument-slot count of the first observed window because ranking precedes synthesis. Both appear only in a ranking heuristic — no admission decision depends on either, and the effect catalog is consulted where it is load-bearing, in qualification and in the runtime facade.

D Paired Statistics for the Primary Families

Per-record paired differences (compiled minus baseline) on the two families that admit a deep prefix, with 10,000-sample bootstrap intervals and two-sided Wilcoxon signed-rank tests over $n = 30$ paired held-out records:

Family	Endpoint	Paired difference (95% CI)	p
	Total tokens	−2196.7 [−2211.1, −2182.5]	1.7×10^{-6}
PR-outcome	Wall latency	−3889 ms [−4499, −3346]	1.9×10^{-9}
	Estimated cost	$-\$5.11 \times 10^{-4}$	1.7×10^{-6}
	Total tokens	−2315.0 [−2366.0, −2224.1]	1.7×10^{-6}
Backlog	Wall latency	−4428 ms [−5400, −3558]	1.9×10^{-9}
	Estimated cost	$-\$5.44 \times 10^{-4}$	1.7×10^{-6}

Provider-request differences are deterministic by construction (every pair removes the same number of turns), so no rank test is reported for them: the statistic would measure the degeneracy rather than the effect.

E Additional Experimental Detail

Primary workflow families. The three-family result is the main paper's strongest live evidence because it combines distinct tool vocabularies, exact task graders, balanced held-out cohorts, and provider-backed execution. The issue-type family intentionally retains a shallower two-read prefix; the newer pull-request and backlog families admit deeper pre-model compaction. This difference explains the lower resource savings on the first family and is part of the evidence boundary, not noise to average away.

Cross-repository extension. The cross-repository study is narrower than the primary workflow families. Its task uses two read-only tools and a simpler exact outcome contract, which is precisely why a fixed template can remain tied or even more efficient than the learned artifact. We keep the result in the main text because it addresses a reviewer-critical scope question—whether guarded specialization survives beyond one repository—while being explicit about the reduced task complexity.

The position invariant, empirically. An early pilot dispatched a compiled *suffix* region at the entry boundary, violating eq. (2). Because the runtime resolves artifacts before the agent has made the calls the region assumed had already happened, the pilot reordered and duplicated tool calls. The failure is a semantic mismatch between compilation-time position and resolution-time position, not a tuning error, which is why eq. (2) is a hard constraint in the deployed compiler rather than a scoring penalty.

Omitted from the main claim. The repository retains additional studies that are useful scientifically but less suitable for the main narrative:

- a natural-order issue study that exposed a factual continuation miss for a more aggressive three-read artifact,
- exploratory guarded composite synthesis and a follow-up comparison to a fair pre-model manual program,
- a bounded GEPA prompt-optimization study in which GEPA retained its seed, so the combined arm is a replication rather than evidence of interaction, and

Table 5: Disaggregated cross-repository outcomes behind table 3. The core protocol uses four completed repositories and one compile-time retirement; the balanced rerun changes the selected repositories and class mix, so it is reported separately rather than pooled with the core protocol. Reductions compare the compiled condition with the paired unchanged baseline.

Protocol	Repository	Held-out	Exact	Req. ↓	Tokens ↓
Core	huggingface/datasets	30	30/30	44.4%	52.2%
Core	pandas-dev/pandas	30	30/30	44.4%	52.5%
Core	psf/requests	30	30/30	44.4%	52.3%
Core	streamlit/streamlit	30	30/30	44.4%	52.7%
Core	pytorch/pytorch	—	RETIRE	—	—
Balanced	pandas-dev/pandas	60	60/60	66.7%	78.6%
Balanced	psf/requests	60	60/60	66.7%	78.7%
Balanced	pytorch/pytorch	60	60/60	66.7%	78.0%

- a portfolio pilot and controlled demonstration suite for runtime boundary coverage, including workloads where removing turns *increases* cost through prefix-cache fragmentation.

These studies remain important to the broader artifact, but they either use a weaker risk level, address a narrower engineering question, or are explicitly exploratory.

F Evidence Breadth and Boundary Audits

The main text uses the nine-page budget for the registered live studies, the time-forward extension, and the two compiler-measured refusal benchmarks. This appendix makes the retained breadth visible without pooling incomparable cohorts or promoting screened and exploratory work to deployment evidence.

F.1 Repository-level time-forward outcomes

The core result succeeds on four repositories (120/120 held-out pairs) and retains the fifth as a compile-time retirement after 116 exact discovery traces. The balanced rerun has a distinct three-repository cohort and reaches 180/180 held-out pairs. These rows show breadth across repositories, not independent evidence for an average effect: all runs share the same provider family and the same narrow two-read outcome task.

F.2 Continuation audit

The earlier natural-order three-read study is excluded from the registered $\alpha = .05$ headline because it used $\alpha = .10$ and exposed a downstream continuation error. Its retained counterfactual replay nevertheless tests a boundary the exact tool contract cannot see: 17/18 candidate continuations pass unchanged, while issue 6602 has a comment-evidence mismatch. Checked rendering repairs that one case, yielding 18/18 final contracts. This audit makes no provider calls and does not estimate live agent quality; it establishes only that the continuation check caught a concrete failure the compacted tool trace alone would miss.

F.3 External evidence map

Table 6 records every named external benchmark in the retained evidence register and its admissible interpretation.

Obtaining observed results from BFCL. BFCL's reference plans carry no tool results, which is why the plan-format row and the compiler row are listed separately. Its multi-turn categories, however, ship an executable stateful backend: each task publishes an `initial_config` snapshot and the classes it involves, so executing the *official gold plan* through the upstream helper yields observed results, an entry snapshot, and an in-process replay oracle, with no model in the loop. Five preconditions fail the run closed rather than publishing: an undeclared method, any upstream execution

Table 6: Disposition of every retained external benchmark family. Only the three compiler-measurement rows support compiler claims; the remaining rows document why broader benchmark names are not silently converted into GAC results. BFCL appears twice because its reference plans and its executed gold plans are different evidence.

Benchmark	Evidence tier	Retained result / claim boundary
NESTFUL	Compiler measurement	1,415 traces; 24 / 12 / 0 replay; all families retire.
API-Bank	Compiler measurement	212 traces; 0 / 2 / 0 replay; all candidates retire.
BFCL v4 (plans)	Gold-plan checker	200/200 valid plans; plan format only.
BFCL v4 (executed)	Compiler measurement	200 traces, 1,142/1,142 exact re-execution; 3 / 3 / 0 replay; all families retire.
ToolSandbox	Live-provider subset	One simulated scenario; compiler not run.
τ^2/τ^3	Live-provider subset	0/4 pass; compiler not run.
ToolBench	Adapter smoke	Ten fixtures; full data/backend unavailable.
AgentBench	Adapter/preflight	Services and Freebase data gated.
GAIA	Access preflight	Authorization denied; no metric.
SWE-bench Verified	Task adapter	Official run host-gated.
BrowseComp	Live-web subset	1/3 correct, 28 searches; bypassed by the compiler.

Table 7: Gate-behaviour inventory from the retained threshold analysis. A partial frontier would show graded coverage as the threshold changes; none appears among the seven current registered artifacts.

Evidence set	Artifacts	Partial frontiers	What it licenses
Current registered $\alpha = .05$	7	0	Exact per-candidate certificates; all are step-like.
Exploratory or looser-α	3	2	Diagnostic evidence only; not the registered claim.
NESTFUL refusal	12 synthesized	0	$n_{\max} = 26 < 92$; no admission.
API-Bank refusal	2 synthesized	0	$n_{\max} = 8 < 92$; no admission.
BFCL v4 refusal	4 synthesized	0	$n_{\max} = 15 < 92$; no admission, pre-registered.

error, a read-like declaration observed mutating state, a compilable declaration observed advancing the scenario RNG, and any inexact re-execution. All held: 200 tasks and 1,142 calls executed cleanly and a second independent pass reproduced every result byte for byte, executing turn-at-a-time where the first pass executed call-at-a-time.

Why the catalog is signed rather than inferred. The same run audits the declarations empirically by snapshotting every involved instance around every call. It observed 1,060 state-mutating calls over 44 of the 81 methods and 94 calls that advanced the seeded scenario RNG, with zero cases of a read-like declaration mutating state or a compilable declaration advancing the RNG. One case is worth naming: a method whose name and docstring present it as a cost getter mutated its instance's internal lookup on all 36 of its calls and is therefore declared a write. A name- or docstring-derived effect label would have admitted a write into a compiled region. Two further methods are read-like but carry no capabilities, because one derives an age from the host date and the other draws from — and advances — the seeded RNG; they are barriers by declaration and account for 35 suppressed spans. Write barriers suppress 2,802 spans in total, against 45 for entry-schema admissibility, 31 for slot ambiguity, and 5 for partial runs.

F.4 What the registered gate has and has not demonstrated

This audit strengthens the evidentiary record precisely by preserving the negative result: the registered gate currently behaves as a support threshold, not as a demonstrated risk–coverage frontier. The exploratory partial frontiers therefore remain diagnostic rather than evidence for the main claim.

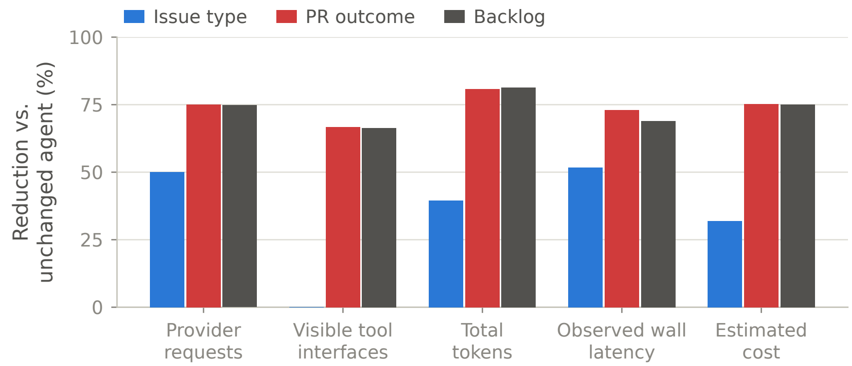

Figure 4: Per-family resource reductions behind table 2, plotted because the spread is the result: admissible prefix depth — not the optimizer — sets the savings, and the two deeper families sit at the feasibility ceiling of eq. (6).

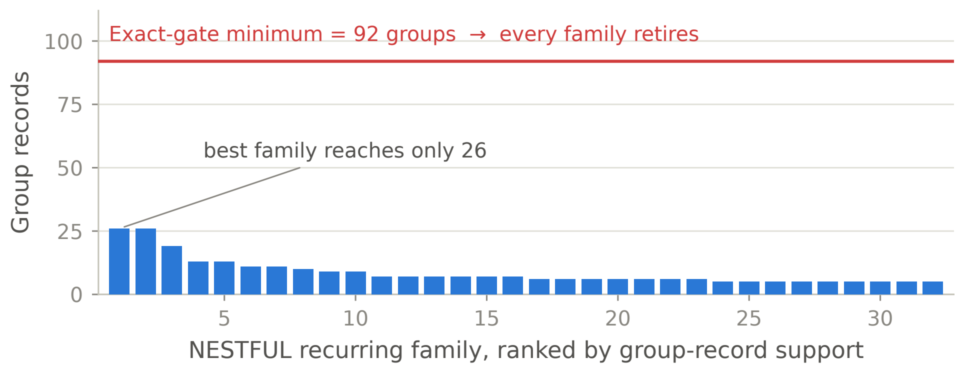

Figure 5: NESTFUL family supports against the 92-group requirement implied by eq. (7) at $\alpha = 0.05$, $\delta = 0.1$, $|\Lambda| = 11$. The largest family reaches 26 groups, so every family retires at calibration even though provenance, synthesis, and replay all succeeded.

G Extended Limitations

Figure 4 plots the per-family spread behind table 2: admissible prefix depth, not the optimizer, sets the savings. The current empirical gate does not yet demonstrate a useful risk–coverage frontier at the registered 5% level. The strongest current runs are mostly all-or-none support tests (fig. 5), and the compiler-wide candidate-search problem is not yet multiplicity-corrected. Likewise, exact task contracts do not certify full semantic equivalence of the final answer, especially once a downstream model continuation is involved. Two further operational caveats belong in the record: the headline live artifact was compiled without a sandbox available to the perturbation suite, so it records `perturbations_claimed: false` rather than a completed challenge, and registry signature verification was configured off in the reported run. These are not hidden defects in the repository; they are the reasons the paper avoids stronger deployment claims.